

```

----- // PreceptorCrud.js
import {
  addPreceptor, getOtrosPreceptoresCollection, deletePreceptorCollection,
  updatePreceptorCollection // getPreceptorCollection
} from "../firebase.js";

/**
 * 1. INICIALIZACIÓN Y TABLA
 */
document.addEventListener("DOMContentLoaded", async () => {
  await mostrarPreceptoresEnHTML();
});

async function mostrarPreceptoresEnHTML() {
  try {
    const tabla = document.querySelector("#tablaPreceptor tbody");
    if (!tabla) return;

    const snapshot = await getOtrosPreceptoresCollection();
    tabla.innerHTML = "";

    if (snapshot.exists()) {
      snapshot.forEach((child) => {
        const id = child.key;
        const preceptor = child.val();
        const fila = document.createElement("tr");
        fila.id = id;
        fila.innerHTML = `
          <td>${preceptor.nombre || "N/A"}</td>
          <td>${preceptor.turno || "N/A"}</td>
          <td>${preceptor.obs || "N/A"}</td>
          <td>
            <button
onclick="window.miModal('detallePreceptorModal','${id}')" class="btn btn-success
btn-sm"><i class="bi bi-binoculars"></i></button>
            <button
onclick="window.miModal('editarPreceptorModal','${id}')" class="btn btn-warning
btn-sm"><i class="bi bi-pencil-square"></i></button>
            <button
onclick="window.miModal('eliminarPreceptorModal','${id}')" class="btn btn-danger
btn-sm"><i class="bi bi-trash"></i></button>
          </td>`;
        tabla.appendChild(fila);
      });
    }
  } catch (error) {
    console.error("Error al cargar preceptor:", error);
  }
}

/**
 * 2. GESTIÓN DE MODALES PARA PRECEPTORES
 */
window.miModal = async function (idModal, idRef = "") {

```

```

try {
  // Limpieza de modales previos
  const existing = document.getElementById(idModal);
  if (existing) {
    const ins = bootstrap.Modal.getInstance(existing);
    if (ins) ins.hide();
    existing.remove();
  }
  document.querySelectorAll('.modal-backdrop').forEach(el => el.remove());

  const rutas = {
    "agregarPreceptorModal": "modales/modalAddPreceptor.php",
    "detallePreceptorModal": "modales/modalDetallesPreceptor.php",
    "editarPreceptorModal": "modales/modalEditarPreceptor.php",
    "eliminarPreceptorModal": "modales/modalDeletePreceptor.php"
  };

  if (!rutas[idModal]) return;

  const response = await fetch(rutas[idModal]);
  const text = await response.text();
  const parser = new DOMParser();
  const htmlDoc = parser.parseFromString(text, 'text/html');
  const modalElement = htmlDoc.querySelector('.modal');
  document.body.appendChild(modalElement);

  const myModal = new bootstrap.Modal(modalElement);
  myModal.show();

  // Cargas específicas según el modal
  if (idModal === "agregarPreceptorModal") await
cargarPreceptoresEnModal();
  if (idModal === "detallePreceptorModal") await
cargarDetallePreceptor(idRef);
  if (idModal === "editarPreceptorModal") await
getPreceptorUpdateCollection(idRef);

  if (idModal === "eliminarPreceptorModal") {
    const btn = modalElement.querySelector("#confirmDeleteBtn");
    if (btn) btn.onclick = async () => {
      await eliminarPreceptor(idRef);
      myModal.hide();
    };
  }
} catch (error) { console.error("Error Modal Preceptor:", error); }
};

/**
 * 3. FUNCIONES DE APOYO
 */
async function cargarPreceptoresEnModal() {
  const select = document.getElementById("selectPreceptorModal");
  if (!select) return;
  const snap = await getPreceptoresCollection();

```

```

        select.innerHTML = '<option selected value="">Seleccione
Preceptor</option>';
        if (snap.exists()) {
            snap.forEach(doc => {
                select.innerHTML += `<option
value="${doc.key}">${doc.val().nombre}</option>`;
            });
        }
    }

    async function cargarDetallePreceptor(id) {
        try {
            const snapshot = await getPreceptorCollection(id);
            if (snapshot.exists()) {
                const data = snapshot.val();
                const contenedor =
document.querySelector("#detallePreceptorContenido ul");
                if (!contenedor) return;
                contenedor.innerHTML = `
                    <li class="list-group-item"><b>Apellido y Nombre: </b>
${data.nombre || "N/A"}</li>
                    <li class="list-group-item"><b>Turno:</b> ${data.turno ||
"N/A"}</li>
                    <li class="list-group-item"><b>Obs: </b> ${data.obs ||
"N/A"}</li>`;
            }
        } catch (error) { console.error("Error detalles preceptor:", error); }
    }

    async function getPreceptorUpdateCollection(id) {
        try {
            const preceptorDoc = await getPreceptorCollection(id);
            if (preceptorDoc.exists()) {
                const data = preceptorDoc.val();
                if (document.querySelector("#nombre"))
document.querySelector("#nombre").value = data.nombre || "";
                if (document.querySelector("#turno"))
document.querySelector("#turno").value = data.turno || "";
                if (document.querySelector("#obs"))
document.querySelector("#obs").value = data.obs || "";
            }
        } catch (error) { console.error("Error precarga edición:", error); }
    }

    /**
     * 4. ACCIONES (CREAR, ACTUALIZAR, ELIMINAR)
     */
    window.addPreceptor = async function (event) {
        event.preventDefault();

        const fd = new FormData(form);

        try {
            await addNuevoPreceptor(fd.get("nombre"), fd.get("turno"),

```

```

fd.get("obs"));
    bootstrap.Modal.getInstance(form.closest('.modal')).hide();
    await mostrarPreceptorsEnHTML();
    window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Preceptor
creado!" });
    } catch (e) { console.error(e); }
};

window.actualizarPreceptor = async function (event) {
    event.preventDefault();
    const formulario = document.querySelector("#formularioPreceptorEdit");
    const formData = new FormData(formulario);
    const data = Object.fromEntries(formData.entries());
    try {
        const camposUpdate = {
            preceptor: data.nombre,
            ubicacion: data.turno,
            obs: data.obs
        };
        await updatePreceptorCollection(data.idPreceptor, camposUpdate);

        bootstrap.Modal.getInstance(document.getElementById("editarPreceptorModal")).hide();
        window.mostrarAlerta({ tipoToast: "success", mensaje: "¡Preceptor
actualizado!" });
        await mostrarPreceptorsEnHTML();
    } catch (error) { console.error("Error al actualizar preceptor:", error); }
};

async function eliminarPreceptor(id) {
    try {
        await deletePreceptorCollection(id);
        window.mostrarAlerta({ tipoToast: "success", mensaje: "Preceptor
eliminado correctamente" });
        await mostrarPreceptorsEnHTML();
    } catch (error) { console.error("Error al borrar preceptor:", error); }
}

-----
// firebase.js - ARCHIVO UNIFICADO
import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.13.0/firebase-app.js";
import {
    getDatabase,
    ref,
    push,
    get,
    update,
    remove,
    query,
    orderByChild
} from "https://www.gstatic.com/firebasejs/10.13.0/firebase-database.js";

const firebaseConfig = {
    apiKey: "AIzaSyAfEDzd8wc6t9Y3foI2HDrWVL_MIzhYnA",

```

```

    authDomain: "asistencia-93328.firebaseio.com",
    databaseURL: "https://asistencia-93328-default-rtdb.firebaseio.com",
    projectId: "asistencia-93328",
    storageBucket: "asistencia-93328.firebaseio.com",
    messagingSenderId: "692275978617",
    appId: "1:692275978617:web:5579ef6c0aeb2d58c7cfa8"
  };

// Inicialización única
export const app = initializeApp(firebaseConfig);
export const db = getDatabase(app);

// Nodos / Colecciones
const coleccionCursos = "cursos";
const coleccionPreceptores = "preceptor"; //para llenar el modal de cursos con
los preceptores existentes
const nodoAlumnos = "tbl_alumnos";

/**
 * --- SECCIÓN CURSOS ---
 */
export const addCurso = (curso, ubicacion, capacidad, obs, preceptorID) =>
  push(ref(db, coleccionCursos), { curso, ubicacion, capacidad, obs,
preceptorID });

export const getCursosCollection = () => get(ref(db, coleccionCursos));

export const getCursoCollection = (id) => get(ref(db,
`${coleccionCursos}/${id}`));

export const updateCursoCollection = (id, newFields) =>
  update(ref(db, `${coleccionCursos}/${id}`), newFields);

export const deleteCursoCollection = (id) =>
  remove(ref(db, `${coleccionCursos}/${id}`));

export const getPreceptoresCollection = () => get(ref(db,
coleccionPreceptores)); //Es la función que trae la lista para el option del
modal de Cursos).

/**
 * --- SECCIÓN ALUMNOS ---
 */
export const addalumno = (curso, nombre, dni, obs, huellaId) => {
  return push(ref(db, nodoAlumnos), { curso, nombre, dni, obs, huellaId,
borrar: "No" });
};

export const getalumnosCollection = () => {
  const consultaOrdenada = query(ref(db, nodoAlumnos), orderByChild('dni'));
  return get(consultaOrdenada);
};

```

```

export const getalumnoCollection = (id) => get(ref(db, `${nodoAlumnos}/${id}`));

export const updatealumnoCollection = (id, newFields) =>
  update(ref(db, `${nodoAlumnos}/${id}`), newFields);

export const deletealumnoCollection = (id) =>
  remove(ref(db, `${nodoAlumnos}/${id}`));

export const marcarParaBorrar = (id) =>
  update(ref(db, `${nodoAlumnos}/${id}`), { obs: "Borrar" });

/**
 * --- SECCIÓN ASISTENCIAS ---
 */
export const getAsistenciasHoy = () => get(ref(db, "asistencia"));

/**
 * --- SECCIÓN PRECEPTORES ---
 */
// Función para guardar (Aseguramos que reciba strings)
export const addPreceptor = (nombre, turno, obs) =>
  push(ref(db, coleccionPreceptores), { nombre, turno, obs });

// Función para listar
export const getOtrosPreceptoresCollection = () => get(ref(db,
coleccionPreceptores));

// Función para eliminar
export const deletePreceptorCollection = (id) => remove(ref(db,
`${coleccionPreceptores}/${id}`));

export const updatePreceptorCollection = (id, newFields) =>
  update(ref(db, `${coleccionPreceptores}/${id}`), newFields);

```